Summary: Dynamo

Summary

1. Dynamo is a highly available **key-value store** developed by **Amazon** for their internal use.
2. Dynamo shows how business requirements can drive system designs. Amazon has chosen to sacrifice strong consistency for **higher availability** based on their business requirements.
3. Dynamo was designed with the understanding that system/hardware failures can and do occur.
4. Dynamo is a **peer-to-peer** distributed system, i.e., it does not have any leader or follower nodes. All nodes are equal and have the same set of roles and responsibilities. This also means that there is **no single point of failure**.
5. Dynamo uses the **Consistent Hashing** algorithm to distribute the data among nodes in the cluster automatically.
6. Data is replicated across nodes for fault tolerance and redundancy. Dynamo replicates writes to a **sloppy quorum** of other nodes in the system instead of a strict majority quorum.
7. For anti-entropy and to resolve conflicts, Dynamo uses **Merkle trees**.
8. Different storage engines can be plugged into Dynamo's local storage.
9. Dynamo uses the **gossip protocol** for inter-node communication.
10. Dynamo makes the system "always writeable" by using **hinted handoff**.
11. Dynamo's design philosophy is to ALWAYS allow writes. To support this, Dynamo allows concurrent writes. Writes can be performed by different servers concurrently, resulting in multiple versions of an object. Dynamo attempts to track and reconcile these changes using **vector clocks**. When Dynamo cannot reconcile an object's state from its vector clocks, it sends it to the client application for reconciliation (*the thought being that the clients have more semantic information on the object and may be able to reconcile it*).
12. Dynamo is able to successfully pull together several distributed techniques such as consistent hashing, p2p, gossip, vector clocks, and quorum, and combine them into a complex system.
13. Amazon built Dynamo for internal use only, so **no security** related issues were considered.

The following table presents a summary of the list of techniques Dynamo uses and their respective advantages:

Problem	**Technique**	**Advantage**
Partitioning	Consistent Hashing	Incremental Scalability
High availability for writes	Vector clocks with reconciliation during reads	Version size is decoupled from update rates.
Handling temporary failures	Sloppy Quorum and Hinted Handoff	Provides high availability and durability guarantee when some of the replicas are not available
Recovering from permanent failures	Anti-entropy using Merkle trees	Synchronizes divergent replicas on the background
Membership and failure detection	Gossip-based membership protocol and failure detection	Preserves symmetry and avoid centralized monitoring

System design patterns

Here is a summary of system design patterns used in Dynamo:

- **Consistent Hashing**: Dynamo uses Consistent Hashing to distribute its data across nodes.
- **Quorum**: To ensure data consistency, each Dynamo write operation can be configured to be successful only if the data has been written to at least a quorum of replica nodes.
- **Gossip protocol**: Dynamo uses gossip protocol that allows each node to keep track of state information about the other nodes in the cluster.
- **Hinted Handoff**: Dynamo nodes use Hinted Handoff to remember the write operation for failing nodes.
- **Read Repair**: Dynamo uses 'Read Repair' to push the latest version of the data to nodes with the older versions.
- **Vector clocks**: To reconcile concurrent updates on an object Dynamo uses Vector clocks.
- **Merkle trees**: For anti-entropy and to resolve conflicts in the background, Dynamo uses Merkle trees.

References and further reading

- Amazon's Dynamo
- Eventually Consistent
- Bigtable
- DynamoDB
- CRDT
- A Decade of Dynamo
- Riak
- Dynamo Architecture
- Dynamo: A flawed architecture

← Previous
Dynamo Characteristics And Criticism

Next →
Quiz: Dynamo

☐ Mark as Completed